

# Mathematical Framework - RL Trading Agent with Small Language Model

## System Overview

**Pipeline:** Raw Market Data → Feature Engineering → SLM Encoding → RL Agent → Trading Decision → Execution

---

## 1. MARKET DATA PROCESSING

### 1.1 Price Returns Calculation

**Equation:**

$$r_t = \frac{P_t - P_{t-1}}{P_{t-1}} = \frac{\Delta P_t}{P_{t-1}}$$

**Variables:**

- $r_t$  = Return at time  $t$  (dimensionless ratio)
- $P_t$  = Close price at time  $t$  (INR)
- $P_{t-1}$  = Close price at previous time step (INR)

**Why Necessary:** Normalizes price movements across different price levels, making patterns comparable over time.

**Influence on Trading:** Positive returns signal potential uptrend continuation; negative returns indicate downtrend.

**Advantages:**

- Scale-invariant comparison
- Standard metric in finance
- Easy interpretation

**Disadvantages:**

- Assumes continuous compounding
- Sensitive to outliers
- Doesn't capture intraday volatility

### Alternative:

$$r_t^{log} = \ln \left( \frac{P_t}{P_{t-1}} \right)$$

(Log returns are time-additive and more suitable for statistical modeling)

---

## 1.2 Volatility Estimation

### Equation (Standard Deviation):

$$\sigma_t = \sqrt{\frac{1}{n-1} \sum_{i=t-n+1}^t (r_i - \bar{r})^2}$$

### Variables:

- $\sigma_t$  = Volatility at time t
- $n$  = Window size (e.g., 20 days)
- $\bar{r}$  = Mean return over window

**Why Necessary:** Quantifies risk and market uncertainty, crucial for position sizing and risk management.

**Influence on Trading:** High volatility → reduce position size or avoid trading; Low volatility → potential for larger positions.

### Advantages:

- Simple to compute
- Widely understood risk metric
- Direct measure of dispersion

### Disadvantages:

- Assumes normal distribution
- Backward-looking
- Equal weight to all observations

### Alternative (EWMA):

$$\sigma_t^2 = \lambda \sigma_{t-1}^2 + (1 - \lambda) r_t^2$$

where  $\lambda \in (0, 1)$  is decay factor (more weight to recent data)

---

## 1.3 Volume-Weighted Average Price (VWAP)

### Equation:

$$VWAP_t = \frac{\sum_{i=1}^t P_i \cdot V_i}{\sum_{i=1}^t V_i}$$

### Variables:

- $P_i$  = Price at interval  $i$
- $V_i$  = Volume at interval  $i$
- $t$  = Current time interval

**Why Necessary:** Represents the "fair value" of the stock, accounting for volume-weighted transactions.

**Influence on Trading:** Price < VWAP → potential buy signal; Price > VWAP → potential sell signal.

### Advantages:

- Considers volume importance
- Institutional benchmark
- Reduces price manipulation impact

### Disadvantages:

- Resets daily (cumulative)
- Lags in fast-moving markets
- Doesn't predict future prices

**Alternative:** Use Time-Weighted Average Price (TWAP) for equal time intervals without volume weighting.

---

## 2. TECHNICAL INDICATORS

### 2.1 Relative Strength Index (RSI)

#### Equation:

$$RSI_t = 100 - \frac{100}{1 + RS_t}$$

where

$$RS_t = \frac{EMA(\text{gains}, n)}{EMA(\text{losses}, n)}$$

### Variables:

- $RSI_t$  = RSI value at time t (0-100 scale)
- $EMA$  = Exponential Moving Average
- $n$  = Period (typically 14)
- gains = positive price changes
- losses = absolute value of negative price changes

**Why Necessary:** Identifies overbought (>70) and oversold (<30) conditions indicating potential reversals.

**Influence on Trading:**  $RSI > 70 \rightarrow$  consider selling;  $RSI < 30 \rightarrow$  consider buying;  $RSI = 50 \rightarrow$  neutral.

### Advantages:

- Bounded indicator (0-100)
- Clear thresholds
- Works in ranging markets

### Disadvantages:

- Can stay overbought/oversold during strong trends
- Lag in signal generation
- Fixed thresholds may not suit all stocks

**Alternative:** Stochastic Oscillator: where  $C$ =current close,  $L_{14}$ =14-period low,  $H_{14}$ =14-period high

---

## 2.2 Moving Average Convergence Divergence (MACD)

### Equation:

$$MACD_t = EMA_{12}(P_t) - EMA_{26}(P_t)$$

$$Signal_t = EMA_9(MACD_t)$$

$$Histogram_t = MACD_t - Signal_t$$

### Variables:

- $EMA_{12}$  = 12-period exponential moving average
- $EMA_{26}$  = 26-period exponential moving average
- $Signal_t$  = 9-period EMA of MACD (signal line)

**Why Necessary:** Captures momentum changes and trend direction through convergence/divergence of moving averages.

**Influence on Trading:** MACD crosses above Signal → bullish signal; MACD crosses below Signal → bearish signal.

**Advantages:**

- Combines trend and momentum
- Clear crossover signals
- Adaptive to price changes

**Disadvantages:**

- Lagging indicator
- Multiple false signals in choppy markets
- Requires parameter tuning

**Alternative:** Price Rate of Change (ROC):  $ROC_t = \frac{P_t - P_{t-n}}{P_{t-n}} \times 100$

---

## 3. SMALL LANGUAGE MODEL (SLM) ENCODING

### 3.1 Token Embedding

**Equation:**

$$\mathbf{e}_i = \mathbf{W}_e \cdot \text{onehot}(\text{token}_i)$$

**Variables:**

- $\mathbf{e}_i \in \mathbb{R}^{d_{\text{model}}}$  = Embedding vector for token  $i$
- $\mathbf{W}_e \in \mathbb{R}^{d_{\text{model}} \times V}$  = Embedding matrix
- $V$  = Vocabulary size
- $d_{\text{model}}$  = Embedding dimension (e.g., 128, 256)

**Why Necessary:** Converts discrete market observations (tokenized features) into continuous vector representations for neural network processing.

**Influence on Trading:** Quality of embeddings determines how well the model captures feature relationships and patterns.

**Advantages:**

- Captures semantic relationships
- Dimensionality reduction
- Learnable representations

### Disadvantages:

- Requires training data
- High-dimensional parameter space
- May not capture numerical precision

**Alternative:** Directly use numerical features:  $\mathbf{x}_t = [r_t, \sigma_t, RSI_t, MACD_t, \dots]^T$

---

## 3.2 Positional Encoding

### Equation:

$$PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$
$$PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

### Variables:

- $PE$  = Positional encoding matrix
- $pos$  = Position in sequence (time step)
- $i$  = Dimension index
- $d_{model}$  = Model dimension

**Why Necessary:** Injects temporal ordering information since transformers don't inherently understand sequence order.

**Influence on Trading:** Enables model to distinguish between recent and distant market events, crucial for temporal patterns.

### Advantages:

- No learnable parameters
- Handles variable sequence lengths
- Smooth interpolation

### Disadvantages:

- Fixed pattern (not adaptive)
- May not suit all temporal patterns
- High-frequency components for large positions

**Alternative:** Learnable positional embeddings:  $\mathbf{P} \in \mathbb{R}^{T_{max} \times d_{model}}$

---

## 3.3 Self-Attention Mechanism

**Equation:**

$$\text{Attention}(Q, K, V) = \text{softmax} \left( \frac{QK^T}{\sqrt{d_k}} \right) V$$

where:

$$Q = \mathbf{W}_Q \mathbf{X}, \quad K = \mathbf{W}_K \mathbf{X}, \quad V = \mathbf{W}_V \mathbf{X}$$

**Variables:**

- $Q, K, V$  = Query, Key, Value matrices
- $\mathbf{X} \in \mathbb{R}^{T \times d_{\text{model}}} =$  Input sequence
- $\mathbf{W}_Q, \mathbf{W}_K, \mathbf{W}_V \in \mathbb{R}^{d_{\text{model}} \times d_k} =$  Projection matrices
- $d_k$  = Key dimension
- $T$  = Sequence length

**Why Necessary:** Allows model to weigh importance of different time steps, identifying which past market states are most relevant to current decision.

**Influence on Trading:** High attention to recent volatility spike → more cautious trading; attention to previous support level → potential entry point.

**Advantages:**

- Captures long-range dependencies
- Parallel computation
- Interpretable attention weights

**Disadvantages:**

- Quadratic complexity in sequence length:  $O(T^2 \cdot d_{\text{model}})$
- High memory consumption
- May overfit to training patterns

**Alternative:** Linear attention:  $\text{Attention}(Q, K, V) = \phi(Q)(\phi(K)^T V)$  where  $\phi$  is kernel function

---

## 3.4 Layer Normalization

**Equation:**

$$\text{LayerNorm}(\mathbf{x}) = \gamma \odot \frac{\mathbf{x} - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta$$

where:

$$\mu = \frac{1}{d_{model}} \sum_{i=1}^{d_{model}} x_i, \quad \sigma^2 = \frac{1}{d_{model}} \sum_{i=1}^{d_{model}} (x_i - \mu)^2$$

#### Variables:

- $\mathbf{x} \in \mathbb{R}^{d_{model}}$  = Input vector
- $\mu$  = Mean
- $\sigma^2$  = Variance
- $\gamma, \beta$  = Learnable scale and shift parameters
- $\epsilon$  = Small constant for numerical stability (e.g.,  $10^{-5}$ )

**Why Necessary:** Stabilizes training by normalizing activations, preventing gradient vanishing/explosion.

**Influence on Trading:** Improves model convergence → faster adaptation to changing market conditions.

#### Advantages:

- Training stability
- Faster convergence
- Reduces internal covariate shift

#### Disadvantages:

- Additional computation
- May smooth out important outliers
- Batch-independent (unlike BatchNorm)

**Alternative:** RMSNorm:  $\text{RMSNorm}(\mathbf{x}) = \frac{\mathbf{x}}{\sqrt{\frac{1}{d} \sum x_i^2 + \epsilon}} \odot \gamma$

---

## 4. REINFORCEMENT LEARNING FRAMEWORK

### 4.1 Markov Decision Process (MDP) Formulation

#### State Space:

$$s_t = [\mathbf{h}_t, p_t, c_t, \mathbf{f}_t]$$

#### Variables:

- $\mathbf{h}_t \in \mathbb{R}^{d_{model}}$  = SLM hidden state (encoded market context)
- $p_t \in \mathbb{R}$  = Current portfolio value



- $c_t \in \mathbb{Z}$  = Current position (shares held)
- $\mathbf{f}_t \in \mathbb{R}^{n_f}$  = Raw features [price, volume, indicators]

### Action Space:

$$a_t \in \mathcal{A} = -n_{max}, \dots, -1, 0, 1, \dots, n_{max}$$

where  $n_{max}$  = maximum shares to trade per step.

**Why Necessary:** Formal mathematical framework to model sequential decision-making under uncertainty.

**Influence on Trading:** Defines what information agent uses and what actions it can take, directly shaping trading strategy.

### Advantages:

- Principled framework
- Proven theoretical foundations
- Handles delayed rewards

### Disadvantages:

- Assumes Markov property (may not hold)
- Large state/action spaces
- Requires extensive exploration

**Alternative:** Partially Observable MDP (POMDP) with belief states:

$$b_t(s) = P(s_t = s | o_{1:t}, a_{1:t-1})$$

## 4.2 Reward Function

### Equation:

$$r_t = \Delta P n L_t - \lambda_1 |\Delta c_t| - \lambda_2 c_t^2 \sigma_t - \lambda_3 \mathbb{1}_{[c_t \cdot c_{t-1} < 0]}$$

where:

$$\Delta P n L_t = c_{t-1} \cdot (P_t - P_{t-1}) - \text{commission} \cdot |a_t| \cdot P_t$$

### Variables:

- $\Delta P n L_t$  = Change in profit/loss (INR)
- $c_t$  = Position at time t (shares)
- $\Delta c_t = c_t - c_{t-1}$  = Position change
- $\sigma_t$  = Volatility
- $\lambda_1, \lambda_2, \lambda_3$  = Penalty weights

- $\mathbb{1}_{[\cdot]}$  = Indicator function
- commission = Transaction cost (e.g., 0.001)

**Why Necessary:** Encodes trading objectives: maximize profit while controlling risk and transaction costs.

#### Influence on Trading:

- High  $\lambda_1 \rightarrow$  fewer trades (lower transaction costs)
- High  $\lambda_2 \rightarrow$  smaller positions in volatile markets
- High  $\lambda_3 \rightarrow$  avoid direction changes (trend following)

#### Advantages:

- Balances multiple objectives
- Customizable penalties
- Directly incorporates costs

#### Disadvantages:

- Requires careful weight tuning
- May create conflicting incentives
- Sparse rewards in early training

#### Alternative (Sharpe Ratio Reward):

$$r_t = \frac{\text{mean}(\text{returns})}{\text{std}(\text{returns})} = \frac{\bar{r}}{\sigma_r}$$


---

## 4.3 Value Function (Temporal Difference)

#### Equation:

$$V(s_t) = \mathbb{E}_{\pi} \left[ \sum_{k=0}^{\infty} \gamma^k r_{t+k} \mid s_t \right]$$

#### TD Update:

$$V(s_t) \leftarrow V(s_t) + \alpha[r_t + \gamma V(s_{t+1}) - V(s_t)]$$

#### Variables:

- $V(s_t)$  = Value of state  $s_t$
- $\gamma \in (0, 1)$  = Discount factor (e.g., 0.99)
- $\alpha \in (0, 1)$  = Learning rate (e.g., 0.001)
- $\pi$  = Policy (action selection strategy)

**Why Necessary:** Estimates expected cumulative reward from a state, guiding agent toward profitable long-term strategies.

**Influence on Trading:** High  $V(s_t) \rightarrow$  favorable market condition, consider entering; Low  $V(s_t) \rightarrow$  unfavorable, consider exiting.

**Advantages:**

- Bootstrapping (faster learning)
- Online learning capability
- Incorporates future expectations

**Disadvantages:**

- Function approximation errors
- Overestimation bias
- Requires extensive sampling

**Alternative (Monte Carlo):**

$$V(s_t) = \frac{1}{N} \sum_{i=1}^N G_t^{(i)}$$

where  $G_t = \sum_{k=0}^{T-t} \gamma^k r_{t+k}$

---

## 4.4 Q-Function (Action-Value)

**Equation:**

$$Q(s_t, a_t) = \mathbb{E}_{\pi} \left[ \sum_{k=0}^{\infty} \gamma^k r_{t+k} \mid s_t, a_t \right]$$

**Bellman Optimality:**

$$Q^*(s_t, a_t) = \mathbb{E}[r_t + \gamma \max_{a'} Q^*(s_{t+1}, a')]$$

**Variables:**

- $Q(s_t, a_t)$  = Value of taking action  $a_t$  in state  $s_t$
- $Q^*$  = Optimal Q-function
- $\max_{a'}$  = Maximum over all possible next actions

**Why Necessary:** Evaluates specific trading actions, enabling comparison and selection of best trades.

**Influence on Trading:** Agent selects action:  $a_t^* = \arg \max_a Q(s_t, a)$ , i.e., trade with highest expected return.

**Advantages:**

- Direct action evaluation
- Off-policy learning possible
- Rich information for decision-making

**Disadvantages:**

- Curse of dimensionality
- Overestimation in function approximation
- Large action spaces challenging

**Alternative (Advantage Function):**

$$A(s_t, a_t) = Q(s_t, a_t) - V(s_t)$$

(measures relative advantage of action)

---

## 4.5 Deep Q-Network (DQN) Loss

**Equation:**

$$\mathcal{L}(\theta) = \mathbb{E}_{(s,a,r,s') \sim \mathcal{D}} \left[ \left( r + \gamma \max_{a'} Q(s', a'; \theta^-) - Q(s, a; \theta) \right)^2 \right]$$

**Variables:**

- $\theta$  = Neural network parameters (current)
- $\theta^-$  = Target network parameters (frozen)
- $\mathcal{D}$  = Replay buffer
- $(s, a, r, s')$  = Experience tuple

**Why Necessary:** Training objective for neural network to approximate optimal Q-function using experience replay.

**Influence on Trading:** Minimizing loss → better action-value estimates → improved trading decisions.

**Advantages:**

- Breaks temporal correlation (replay buffer)
- Stabilizes training (target network)
- Sample efficient

### Disadvantages:

- Overestimation bias
- Hyperparameter sensitive
- Memory intensive

### Alternative (Double DQN):

$$y_t = r_t + \gamma Q(s_{t+1}, \arg \max_{a'} Q(s_{t+1}, a'; \theta); \theta^-)$$

(uses online network to select action, target network to evaluate)

---

## 4.6 Policy Gradient (REINFORCE)

### Equation:

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\pi_{\theta}} \left[ \sum_{t=0}^T \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \cdot G_t \right]$$

where:

$$G_t = \sum_{k=0}^{T-t} \gamma^k r_{t+k}$$

### Variables:

- $J(\theta)$  = Expected cumulative reward
- $\pi_{\theta}(a|s)$  = Policy (probability of action a in state s)
- $G_t$  = Return from time t
- $\nabla_{\theta}$  = Gradient with respect to parameters

**Why Necessary:** Directly optimizes policy to maximize expected returns, suitable for continuous action spaces.

**Influence on Trading:** Updates policy to favor actions that led to profitable trades, refining strategy over time.

### Advantages:

- Direct policy optimization
- Handles continuous actions
- Can learn stochastic policies

### Disadvantages:

- High variance

- Sample inefficient
- Slow convergence

#### Alternative (Actor-Critic):

$$\nabla_{\theta} J(\theta) = \mathbb{E} [\nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \cdot A(s_t, a_t)]$$

(uses advantage function as baseline to reduce variance)

---

## 4.7 Proximal Policy Optimization (PPO)

#### Equation:

$$\mathcal{L}^{CLIP}(\theta) = \mathbb{E}_t [\min(r_t(\theta) A_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) A_t)]$$

where:

$$r_t(\theta) = \frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_{old}}(a_t | s_t)}$$

#### Variables:

- $r_t(\theta)$  = Probability ratio
- $A_t$  = Advantage estimate
- $\epsilon$  = Clipping parameter (e.g., 0.2)
- $\text{clip}(x, a, b)$  = Clamps x to [a, b]

**Why Necessary:** Prevents destructively large policy updates, ensuring stable training in volatile markets.

**Influence on Trading:** Conservative policy updates → gradual strategy refinement → more robust to market regime changes.

#### Advantages:

- Training stability
- Sample efficient
- Simple to implement

#### Disadvantages:

- Hyperparameter tuning needed
- May be conservative in exploration
- Clipping heuristic

**Alternative (TRPO):** Uses KL divergence constraint:  $\mathbb{E}[KL(\pi_{old} || \pi)] \leq \delta$

---

## 5. PORTFOLIO MANAGEMENT

### 5.1 Position Sizing (Kelly Criterion)

**Equation:**

$$f^* = \frac{p(b+1) - 1}{b}$$

**Simplified for trading:**

$$f^* = \frac{\mu - r_f}{\sigma^2}$$

**Variables:**

- $f^*$  = Optimal fraction of capital to invest
- $p$  = Probability of winning
- $b$  = Odds (gain/loss ratio)
- $\mu$  = Expected return
- $r_f$  = Risk-free rate
- $\sigma^2$  = Variance of returns

**Why Necessary:** Determines optimal position size to maximize long-term capital growth while managing risk.

**Influence on Trading:** High Sharpe ratio → larger positions; High volatility → smaller positions.

**Advantages:**

- Mathematically optimal
- Accounts for risk-return tradeoff
- Prevents over-leveraging

**Disadvantages:**

- Assumes accurate parameter estimates
- Can be aggressive (use fractional Kelly)
- Requires stationary statistics

**Alternative (Fixed Fractional):**

$$\text{Position Size} = \frac{\text{Risk per Trade}}{\text{Stop Loss Distance}}$$

---

### 5.2 Value at Risk (VaR)

### Equation (Parametric):

$$VaR_{\alpha} = \mu + \sigma \cdot \Phi^{-1}(\alpha)$$

### Variables:

- $VaR_{\alpha}$  = Maximum loss at confidence level  $\alpha$  (e.g., 0.05)
- $\mu$  = Mean return
- $\sigma$  = Standard deviation
- $\Phi^{-1}$  = Inverse cumulative distribution function

**Why Necessary:** Quantifies downside risk, helping set stop-loss levels and risk limits.

**Influence on Trading:** If current loss approaches VaR → close position to limit maximum loss.

### Advantages:

- Single risk number
- Widely used standard
- Easy to communicate

### Disadvantages:

- Doesn't capture tail risk
- Assumes normal distribution
- Backward-looking

### Alternative (Conditional VaR / CVaR):

$$CVaR_{\alpha} = \mathbb{E}[L | L > VaR_{\alpha}]$$

(expected loss given loss exceeds VaR)

---

## 6. EXECUTION & OPTIMIZATION

### 6.1 Slippage Estimation

#### Equation:

$$\text{Slippage} = \alpha \cdot |a_t| + \beta \cdot \frac{|a_t|}{V_t}$$

#### Variables:

- $\alpha$  = Fixed slippage per share (INR)
- $\beta$  = Market impact coefficient



- $a_t$  = Order size (shares)
- $V_t$  = Average volume

**Why Necessary:** Accounts for execution costs beyond commissions, especially for large orders.

**Influence on Trading:** Large orders in low-volume stocks incur high slippage → split into smaller orders or avoid.

#### Advantages:

- Realistic cost modeling
- Guides order sizing
- Improves backtest accuracy

#### Disadvantages:

- Parameters hard to estimate
- Varies by market conditions
- Non-linear in practice

#### Alternative (Square-root Model):

$$\text{Market Impact} = \gamma \sigma \sqrt{\frac{|a_t|}{V_t}}$$

where  $\gamma$  is coefficient

## 6.2 Execution Algorithm (TWAP)

#### Equation:

$$a_i = \frac{A}{N}, \quad t_i = t_0 + i \cdot \frac{T}{N}$$

#### Variables:

- $A$  = Total order size
- $N$  = Number of slices
- $a_i$  = Size of slice  $i$
- $t_i$  = Time of slice  $i$
- $T$  = Total execution window

**Why Necessary:** Reduces market impact by distributing large orders over time.

**Influence on Trading:** RL agent outputs target position → TWAP breaks it into smaller orders → executed gradually.

**Advantages:**

- Simple implementation
- Reduces price impact
- Predictable execution

**Disadvantages:**

- Ignores market dynamics
- May miss opportunities
- Vulnerable to momentum

**Alternative (VWAP):** Execute proportional to historical intraday volume profile.

---

## 7. COMPLETE PIPELINE EQUATIONS

### Step-by-Step Mathematical Flow:

**1. Data Input → Feature Engineering:**

$$\mathbf{f}_t = [r_t, \sigma_t, RSI_t, MACD_t, Volume_t, \dots]^T$$

**2. SLM Encoding:**

$$\mathbf{h}_t = \text{SLM}(\mathbf{f}_{t-T:t}) = \text{TransformerEncoder}(\mathbf{E} + \mathbf{PE})$$

**3. State Construction:**

$$\mathbf{s}_t = [\mathbf{h}_t, p_t, c_t, \mathbf{f}_t]$$

**4. Action Selection:**

$$a_t = \arg \max_a Q(s_t, a; \theta) + \epsilon_t$$

(with exploration noise  $\epsilon_t$ )

**5. Environment Step:**

$$s_{t+1}, r_t = \text{TradingEnv.step}(a_t)$$

**6. Experience Storage:**

$$\mathcal{D} \leftarrow \mathcal{D} \cup (s_t, a_t, r_t, s_{t+1})$$

**7. Policy Update:**

$$\theta \leftarrow \theta - \alpha \nabla_{\theta} \mathcal{L}(\theta)$$

8. Order Execution:

Execute(TWAP( $a_t, T, N$ ))

Summary Table: Key Equations by Purpose

| Purpose          | Equation                                                          | Key Parameters        |
|------------------|-------------------------------------------------------------------|-----------------------|
| Risk Measure     | $\sigma = \sqrt{\frac{1}{n} \sum (r_i - \bar{r})^2}$              | $n$ (window)          |
| Market Signal    | $RSI = 100 - \frac{100}{1+RS}$                                    | period = 14           |
| Context Encoding | $\text{Attn}(Q, K, V) = \text{softmax}(\frac{QK^T}{\sqrt{d_k}})V$ | $d_k$ (dimension)     |
| Value Learning   | $V(s) \leftarrow V(s) + \alpha[r + \gamma V(s') - V(s)]$          | $\alpha, \gamma$      |
| Policy Learning  | $\nabla J = \mathbb{E}[\nabla \log \pi \cdot A]$                  | learning rate         |
| Position Sizing  | $f^* = \frac{\mu - r_f}{\sigma^2}$                                | risk tolerance        |
| Risk Limit       | $VaR = \mu + \sigma \Phi^{-1}(\alpha)$                            | $\alpha$ (confidence) |

Critical Hyperparameters

- SLM:**  $d_{model} = 256$ , layers=4, heads=8
- RL:**  $\gamma = 0.99$ ,  $\alpha = 10^{-4}$ , buffer size= $10^6$
- Trading:** commission=0.001, max position=1000 shares
- Risk:**  $\lambda_1 = 0.001$ ,  $\lambda_2 = 0.01$ ,  $\lambda_3 = 0.05$